# Übung 1 · Das Modell als Baustein

**Cloud-native AI · WS 2026/27 · Praktikumsabgabe P1**

- **Start:** Übung am Dienstag, 13.10.2026, 11:30, O/201
- **Abgabe:** Montag, 19.10.2026, 23:59, in Moodle
- **Arbeitsform:** einzeln oder zu zweit, wie in der Moodle-Gruppeneinteilung eingetragen
- **Aufwand:** etwa 6 Stunden für den Kern, dazu etwa 2 Stunden für den Zusatz

---

## Worum es geht

In dieser Übung legt ihr das Fundament für euer System im Praktikum: eine kleine Anwendung, die ein Sprachmodell aufruft, ohne zu wissen, welches. Das Modell steckt hinter einer Schnittstelle und wird nur über Konfiguration ausgewählt. Danach macht ihr sichtbar, was in VL 2 Theorie war: Wie stark streuen Antworten bei verschiedenen Temperaturen, was kostet Deutsch im Vergleich zu Englisch, und wie bringt ihr ein Modell dazu, verlässlich Daten statt Fließtext zu liefern?

Am Ende steht der erste Baustein des Campus-Assistenten: Er ordnet Mails von Studierenden strukturiert ein.

Grundlage sind die Skripte zu **VL 1** (Was ist Cloud-native AI?) und **VL 2** (Foundation Models verstehen).

| Teil | Inhalt | Aufwand | |
|---|---|---|---|
| **1** | Setup und erster Modellaufruf | ca. 1 h | Kern |
| **2** | Modell austauschen, nur über Konfiguration | ca. 45 min | Kern |
| **3** | Sampling und Tokens messen | ca. 1 h | Kern |
| **4** | Anfragen strukturiert einordnen | ca. 2,5 h | Kern |
| **5** | README mit Ergebnissen und Reflexion | ca. 45 min | Kern |
| **Z** | Zusatz: Ausgaben inhaltlich absichern | ca. 2 h | Zusatz |

Das Starter-Projekt liegt im Ordner `starter/`.

---

## Teil 1 · Setup und erster Modellaufruf (Kern)

1. **Docker** installieren, falls noch nicht vorhanden (Docker Desktop oder Docker Engine). Test: `docker run --rm hello-world`
2. Ein **eigenes Git-Repository** anlegen und das Starter-Projekt hineinkopieren. Bei Zweiergruppen arbeitet ihr in einem gemeinsamen Repository.
3. `.env.example` nach `.env` kopieren. Die Standardwerte nutzen das lokale Modell, ihr müsst nichts ändern.
4. Starten:

```
docker compose up --build app
```

Beim ersten Start lädt der Dienst `modell-laden` zwei Modelle herunter: `llama3.2:3b` (ca. 2 GB) und `qwen2.5:0.5b` (ca. 400 MB). Das dauert einmalig ein paar Minuten. Danach beantwortet die Anwendung eine Standardfrage und gibt Messwerte aus.

Eigene Frage stellen. Der Ordner `app/` ist in den Container eingebunden, Änderungen am Code wirken also sofort ohne neuen Build:

```
docker compose run --rm app hallo_llm.py "Wie melde ich mich zu einer Prüfung an?"
```

**Aufgaben:**

1. Lest `compose.yaml`, `app/llm.py` und `app/hallo_llm.py`. Beschreibt im README in drei, vier Sätzen: Welche Container laufen, und wie findet die Anwendung das Modell?
2. Stellt drei verschiedene Fragen an den Campus-Assistenten. Notiert jeweils **Time to First Token** und **Gesamtdauer**.

---

## Teil 2 · Modell austauschen (Kern)

Ändert in `.env` nur die Zeile `LLM_MODEL` auf `qwen2.5:0.5b` und stellt dieselben drei Fragen noch einmal. **Am Code ändert ihr nichts.**

**Aufgaben:**

1. Füllt im README die Tabelle aus:

| Frage | Modell | TTFT (ms) | Gesamt (ms) | Antwort brauchbar? |
|---|---|---|---|---|
| … | llama3.2:3b | | | |
| … | qwen2.5:0.5b | | | |

2. Fragt beide Modelle: „Ist die HTWG Konstanz eine private oder eine staatliche Hochschule?“ Was fällt euch auf?

**Optional, wenn die Academic Cloud erreichbar ist:** Mit einem API-Key über KISSKI tragt ihr Variante B aus `.env.example` ein und nutzt `qwen3-30b-a3b-instruct-2507` als drittes Modell im Vergleich. Wer keinen Key hat oder wenn der Dienst nicht läuft, verliert dadurch nichts. Alle Teile funktionieren komplett lokal.

---

## Teil 3 · Sampling und Tokens messen (Kern)

Stellt `LLM_MODEL` zurück auf `llama3.2:3b`.

### Sampling

```
docker compose run --rm app sampling.py
```

Das Skript stellt dieselbe Frage je fünfmal bei Temperature 0, 0,7 und 1,2 und zählt die verschiedenen Antworten.

1. Wie viele verschiedene Antworten bekommt ihr bei jeder Temperature?
2. Liefert Temperature 0 wirklich fünfmal dieselbe Antwort? Wenn nicht: Woran könnte das liegen? (Skript VL 2, Abschnitt „Inkonsistenz“.)
3. Startet das Skript mit einer Faktenfrage, z. B. `sampling.py "Ist die HTWG Konstanz eine private oder staatliche Hochschule? Antworte mit einem Satz."`. Achtet nicht nur auf „privat oder staatlich“, sondern auf jedes Detail, etwa wofür die Abkürzung HTWG steht. Welche Fehler kommen schon bei Temperature 0 vor, welche erst bei höheren Werten? Was sagt das über die Ursache (Sampling oder Trainingsdaten)?

### Tokens

```
docker compose run --rm app tokens.py
```

1. Wie viele Tokens braucht der deutsche Text, wie viele der englische? Um wie viel Prozent ist Deutsch teurer?
2. Ersetzt in `tokens.py` die Texte durch eine eigene Frage mit langen zusammengesetzten Wörtern (z. B. „Prüfungsanmeldungsfrist“). Wird der Unterschied größer?

---

## Teil 4 · Anfragen strukturiert einordnen (Kern)

Der Campus-Assistent soll jede Mail zuerst einordnen: Um welches **Anliegen** geht es (Prüfung, Mensa, Bibliothek, Sonstiges), welcher **Studiengang**, welche **Frist**? Das Ergebnis verarbeitet ein Programm weiter. Es muss also immer dieselbe Form haben.

Öffnet `app/einordnen.py`. Die Klasse `Anfrage` und die Hilfsfunktion `schema_von` sind vorgegeben. Eure Aufgaben stehen an den `TODO`-Kommentaren:

1. **TODO 1:** Ruft das Modell mit `response_format=schema_von(Anfrage)` auf. Wählt eine passende Temperature und begründet sie im Code mit einem Kommentar.
2. **TODO 2:** Prüft die Antwort mit `Anfrage.model_validate_json(...)`.
3. **TODO 3:** Ist die Antwort ungültig, versucht es erneut, höchstens dreimal. Danach soll das Programm mit einem klaren Fehler abbrechen, statt mit kaputten Daten weiterzuarbeiten.
4. **TODO 4:** Wertet alle Test-Mails aus `mails.json` aus und zählt, wie viele Anliegen richtig erkannt sind.

```
docker compose run --rm app einordnen.py
docker compose run --rm app einordnen.py "Eure eigene Mail"
```

**Aufgaben zur Auswertung:**

1. Wie viele der acht Mails ordnen `llama3.2:3b` und `qwen2.5:0.5b` jeweils richtig ein?
2. Schaut euch die Fehler an. Ist das Anliegen falsch, oder stimmen Studiengang und Frist nicht?
3. Schaut auch bei den „richtigen" Fällen genau hin: Steht bei `studiengang` manchmal der **Text** `"null"` statt eines echten leeren Werts? Nennt das Modell eine **Frist, die gar nicht in der Mail steht**? Warum lässt das Schema beides durch?
4. Die letzte Mail in `mails.json` versucht, das Modell umzulenken. Was passiert? (Darum geht es in VL 3.)
5. Entfernt testweise `response_format` und verlasst euch nur auf den System-Prompt. Gebt die Rohantwort des Modells mit `print` aus. Was geht schief? Warum bringt euer erneuter Versuch bei Temperature 0 nichts, und was könntet ihr ändern? Stellt `response_format` danach wieder her.

---

## Teil 5 · README (Kern)

Euer README ist die Abgabe. Es enthält:

1. **Namen** und Matrikelnummern
2. **Start:** den einen Befehl, mit dem man euer System startet, und welche `.env` es braucht
3. **Ergebnisse** aus Teil 1 bis 4: Beschreibung der Container, Tabelle aus Teil 2, Messwerte aus Teil 3, Trefferquote aus Teil 4
4. **Reflexion**, je zwei bis vier Sätze:
    1. Warum steht das Modell in `.env` und nicht im Code?
    2. Welche Temperature habt ihr in `einordnen.py` gewählt, und warum?
    3. Was garantiert `response_format` mit Schema, und was nicht?
    4. Wann würdet ihr ein lokales Modell, wann einen Cloud-Dienst wählen? Nennt je zwei Gründe.

---

## Zusatz · Ausgaben inhaltlich absichern

Dieser Teil **entfällt**, wenn ihr am Semesterende mindestens 80 % Anwesenheit nachweist. Wer nicht sicher ist, gibt ihn mit ab. Zusatzteile sind nicht nachreichbar.

Das Schema garantiert die Form, nicht den Inhalt. In Teil 4 habt ihr gesehen, dass das Modell trotzdem `"null"` als Text schreibt oder Fristen erfindet. Fangt das ab:

1. Ergänzt `Anfrage` um **Pydantic-Validatoren** (`field_validator`):
    - Der Text `"null"`, `"None"` oder eine leere Zeichenkette wird zu einem echten `None`.
    - `studiengang` ist nur gültig, wenn es ein Kürzel aus einer festen Liste ist (z. B. AIN, WIN, MSI, und weitere, die ihr für sinnvoll haltet). Alles andere wird zu `None`.
2. Prüft **im Code, nicht im Modell**, ob die erkannte Frist tatsächlich in der Mail vorkommt. Findet ihr sie nicht, setzt ihr sie auf `None` und gebt eine Warnung aus.
3. Messt die Wirkung: Wie viele der acht Mails haben vorher und nachher **alle drei Felder** richtig? Anliegen und Studiengang vergleicht ihr exakt. Eine Frist zählt als richtig, wenn die erwartete

Frist im Ergebnis enthalten ist, „Freitag, 16.10." passt also zu „16.10.". Tragt beide Zahlen ins README ein.

---

## Was ihr abgebt

**In Moodle**, Aufgabe „Übung 1 (P1)": den **Link zu eurem Repository** und den Commit-Hash eures Abgabestands. Das Repository muss für mich lesbar sein: öffentlich, oder ihr ladet meinen GitHub-Account `PascalLaube` als Collaborator ein.

**Harte Kriterien**, ohne die die Abgabe nicht bestanden ist:

- ☐ Das System startet mit `docker compose up --build app` nach einer im README beschriebenen `.env`.
- ☐ Keine Secrets im Repository. `.env` steht in `.gitignore` und ist nicht committet. Prüft es mit `git status`.
- ☐ Abhängigkeiten sind gepinnt (`requirements.txt` mit festen Versionen, feste Image-Tags).
- ☐ `einordnen.py` läuft durch und gibt die Trefferquote aus.
- ☐ Das README enthält alle Punkte aus Teil 5.
- ☐ Wer unter 80 % Anwesenheit landen könnte: Zusatzteil im README dokumentiert.

---

## Tipps

- **Modelle vorab laden:** Die Modelle sind zusammen etwa 2,4 GB groß. Startet `docker compose up --build app` am besten schon **vor der Übung** zu Hause, sonst teilen sich alle im Raum das WLAN. `docker compose logs modell-laden` zeigt den Fortschritt.
- **Hohe TTFT direkt nach einem Modellwechsel?** Ollama lädt das Modell beim ersten Aufruf in den Speicher. Messt deshalb erst ab dem zweiten Aufruf.
- **Wenig Speicher auf dem Laptop?** Arbeitet mit `qwen2.5:0.5b` und gebt `llama3.2:3b` an, wo ihr es nicht ausführen konntet. Schreibt das ins README.
- **Kein Docker möglich?** Meldet euch bis Donnerstag per Mail, dann finden wir eine Lösung.
- **KI-Werkzeuge** dürft ihr nutzen. Ihr müsst jede Zeile eures Codes erklären können.